

AI Modular System Overview

Purpose:

Provide a modern, intelligent, modular AI backend for a real estate website. Each module is independent, exposes clean APIs, and can be integrated with the frontend/backend team.

Architecture:

- **Recommender Module: Suggests properties based on user input and property features**
- **Analyzer Module: Generates analytical insights about the real estate market**
- **AI Agent (Chatbot) Module: Conversational interface that collects preferences, triggers recommendations, and optionally books appointments**

Design Principles:

- **Modular and independent**
 - **Tool-based orchestration (chatbot can call other modules)**
 - **Controlled complexity (no fully autonomous reasoning agent)**
 - **Professional, demo-ready, and easily integrated with .NET backend**
-

1. Recommender Module

Purpose:

Suggest properties tailored to the user's preferences.

How it works:

- **Uses content-based matching (budget, location, bedrooms, etc.)**
- **Adds lightweight hybrid signals (popularity, recency)**
- **Ranks properties using a weighted scoring formula**

When it runs:

- **Triggered only when the user asks for recommendations (explicitly or implicitly via chatbot).**

Output:

- Top N ranked property suggestions (JSON response).
-

2. Analyzer Module

Purpose:

Provide market insights and trends.

What it provides:

- Average prices by area
- Property distribution by type
- Market trends
- Aggregated statistics

Output:

- Structured data for charts and dashboards on frontend.
-

3. AI Agent (Chatbot) Module

Purpose:

Acts as an intelligent conversational interface.

Responsibilities:

- Detect user intent (search, recommend, book, analyze)
- Extract user preferences (budget, location, etc.)
- Store session context
- Call recommender or analyzer when needed
- Optionally handle booking requests

Important:

The chatbot does not replace backend logic — it acts as a smart orchestrator.

Integration with .NET Website

- Python AI modules run as independent REST APIs (FastAPI).

- .NET backend calls these APIs using HTTP requests.
- AI modules return JSON responses.
- .NET processes the response and sends results to frontend.

This keeps:

- Clear separation of responsibilities
- Clean modular architecture
- Easy future scalability